Manual – Kernel Data Structures

Хуваарилалтын код: 4 · F.CSM302 Үйлдлийн систем · БД1 · Repository: github.com/grl059/OS_BD1

1. Хэрэглэгчийн заавар

1.1 Програмын зорилго

Linux kernel-ийн өгөгдлийн бүтцүүдийг загварчилж, элемент нэмэх, устгах, хайх үйлдэл бүрийн дараах бүтцийг график дээр харуулна. Бүтцүүд: холбоост жагсаалт (`list_head`, struct birthday), queue (FIFO), stack (LIFO), bitmap, hash map (collision-той), хайлтын мод (энгийн BST ба тэнцвэржсэн AVL), процесс ба CPU (PCB, ready queue, FCFS ба Round Robin). Kernel module нь `struct color` жагсаалт болон `start` параметртэй Collatz жагсаалтыг kernel дотор үүсгэнэ. Хэрэглэгч өөрийн огноо, тоо, түлхүүр, процессоо оруулж туршина.

1.2 Дэмжих орчин

Хэсэг	Орчин
Веб апп (web/, Next.js)	Node.js ≥ 18 (`npm install`, `npm run dev`), орчин үеийн browser (Chrome, Edge, Firefox, Safari). Веб апп дэвсгэр дээр `./kds --json`-г дуудаж ажилладаг тул эхлээд C цөмийг build хийсэн байх шаардлагатай (1.3-г үз).
CLI (./kds)	Linux, macOS эсвэл Windows (MSYS2/MinGW эсвэл WSL); gcc ≥ 9 (эсвэл clang), GNU make
Нэмэлт тест	python3 ≥ 3.8, Node.js ≥ 18
Kernel module	Жинхэнэ Linux (VM эсвэл үндсэн машин; WSL/Docker биш), `linux-headers-$(uname -r)`, sudo эрх

1.3 Суулгах эсвэл нээх алхам

1. Repository-г татна: `git clone https://github.com/grl059/OS_BD1.git` эсвэл GitHub-ийн «Code → Download ZIP»-ээр татаж задлана.
2. **CLI (C цөм):** терминалд repository-ийн хавтсанд `make` ажиллуулна. Ингэснээр `./kds` (Windows дээр `kds.exe`) үүснэ. Веб аппд энэ алхам заавал хэрэгтэй.
3. **Веб апп:** `cd web` → `npm install` (нэг удаа) → `npm run dev`. Терминал дээр гарсан хаягийг (жишээ нь `http://localhost:3000`) browser-т нээнэ.

1.4 Оролт ба тохиргоо

Команд	Оролт	Хязгаар
`list` `add_head|add_tail|del|find <д> <с> <ж>`	өдөр, сар, жил	жил 1900–2100, хүчинтэй огноо (2-р сарын 29-г өндөр жилээр шалгана)
`queue|stack push|find` <тоо>, `pop`	бүхэл тоо	багтаамж 8 (overflow/underflow мэдээлнэ)
`bitmap size <n>`, `set|clear|test <i>`	хэмжээ, индекс	n = 1..128, индекс 0..n−1
`hash size <n>`, `put <key> <value>`, `get|del <key>`	bucket-ийн тоо, түлхүүр, утга	n = 1..16 (анхдагч 8); түлхүүр/утга 1–12 тэмдэгт (үсэг, тоо, _); нийт ≤ 24
`tree mode bst|avl`, `insert|delete|find <тоо>`	төрөл, бүхэл тоо	төрлийг зөвхөн хоосон модонд солино; давхардсан утга татгалзана
`proc new <нэр> <ажил>`	нэр, ажлын тик	нэр 1–8 тэмдэгт, ажил 1–20 тик, амьд процесс ≤ 8
`sched policy fcfs | rr` <квант>, `tick|print|clear`	бодлого, квант	квант 1–10; 1 tick = 1 нэгж цаг
`... print`, `... clear`	—	бүтцийн төлөвийг хэвлэх / цэвэрлэх
`insmod kds_module.ko start=N`	Collatz-ийн эхний гишүүн	1..1 000 000 (анхдагч 25)

`#`-ээс хойших хэсэг коммент. Тоог `+5`, `-3`, `007` хэлбэрээр бичиж болно. `3x`, `1.5` хэлбэрийн утга эсвэл int-ийн мужаас хэтэрсэн тоо буруу оролтод тооцогдоно.

1.5 Ажиллуулах, зогсоох, reset хийх

- **Веб апп:** «Тоглох» хуудас дээр зүүн талаас бүтцээ сонгоно. Команд бичих талбарт команд (жишээ нь `list add_tail 2 8 1995`) бичиж Enter дарна, эсвэл жишээ скриптийг сонгож ажиллуулна. **«Эхнээс нь»** товч тухайн бүтцийг reset хийнэ (хуудсыг дахин ачаалах нь бүгдийг reset хийнэ, учир нь бүтцийн төлөв зөвхөн C программ дотор, дэвсгэр process бүрт шинээр эхэлдэг).
- **CLI:** `./kds файл.txt` нь script ажиллуулна. `./kds` гэж ажиллуулаад команд бичиж болно. **Ctrl+D** дарж зогсооно (Ctrl+C мөн зогсооно). Програмыг дахин ажиллуулахад бүх бүтэц хоосон эхэлнэ (reset). Бүтэц тус бүрийг `list clear`, `hash clear`, `sched clear`-ээр цэвэрлэнэ. JSON горимоор ажиллуулахад `./kds --json` гэж нэмж бичнэ.
- **Kernel module:** `sudo insmod` ачаална, `sudo rmmod kds_module` буулгана (2.6).

1.6 Дүрслэлийн тэмдэг, өнгө, төлөв

Веб апп нь тийл-ногоон (teal/green) өнгөний загвартай. Дараах зарчмуудаар бүтцийг зурдаг:

Тэмдэг	Утга
Хайрцаг/дугуйлсан элемент	node (struct birthday, queue/stack-ийн элемент, hash-ийн түлхүүр, PCB)
Өөр өнгөөр тодотгосон элемент	сүүлийн үйлдэлд нэмэгдсэн эсвэл олдсон элемент
Сум (next/prev)	жагсаалтын холбоос (давхар холбоост)
Сум (hash bucket, ready queue)	жагсаалтын дараагийн элемент; bucket-д 2 ба түүнээс олон элемент байвал collision
Дүүргэсэн нүд (1) / хоосон нүд (0)	bitmap-ийн асаалттай / унтраалттай бит
Дугаарлагдсан багана	hash-ийн bucket-ийн индекс
PCB карт, доорх ажлын түүх	процесс ба ажлын явц; тик бүрд ажилласан процесс, «idle» нь CPU сул
Модны доорх «өндөр» бичиг	модны түвшний тоо = хамгийн муу хайлтын алхам
Ажлын түүхэн дэх OK / ERR мөр	OK / ERR мессеж, Монгол тайлбартай

Эдгээр бүх дүрслэл нь веб апп дотор зураг зурж гаргадаг дүрс биш, C программын `--json` горимоос ирсэн төлөвөөс шууд зурагддаг.

1.7 Жишээ

Жишээ 1 (сурах бичгийн жишээ). «Тоглох» хуудас дээр «list» бүтцийг сонгоод доорх командуудыг нэг нэгээр бичиж Enter дарна, эсвэл «Шатлал» хуудсанд байгаа ижил жишээг сонгоно (CLI: `./kds examples/textbook.txt`). Хүлээгдэх үр дүн: 5 огноо дараалаараа жагсаж, устгасны дараа жагсаалт хоосон болно.

```
OK list add_tail 2/8/1995 (size=1)
OK list add_tail 17/3/1998 (size=2)
OK list add_tail 29/2/2000 (size=3)
OK list add_tail 5/11/2001 (size=4)
OK list add_tail 30/12/2003 (size=5)
LIST size=5: 2/8/1995 -> 17/3/1998 -> 29/2/2000 -> 5/11/2001 -> 30/12/2003
OK list del 17/3/1998 (size=4)
LIST size=4: 2/8/1995 -> 29/2/2000 -> 5/11/2001 -> 30/12/2003
OK list del 2/8/1995 (size=3)
OK list del 29/2/2000 (size=2)
OK list del 5/11/2001 (size=1)
OK list del 30/12/2003 (size=0)
LIST size=0: (empty)
```

Жишээ 2 (өөрийн оролт – bitmap). «Тоглох» хуудсанд «bitmap» бүтцийг сонгоод командаар `bitmap size 16`, `bitmap set 3`, `bitmap set 9` гэж оруулна. 16 нүдний 3 ба 9-р нүд

тодотгогддог болно. `bitmap print` командын гаралт: `BITMAP n=16: 0001000001000000 (set=2).` `bitmap set 16` ГЭВЭЛ `ERR bitmap: index 16 out of range 0..15` гарна.

Жишээ 3 (процесс ба CPU). «Тоглох» хуудсанд «sched»/«proc» бүтцийг сонгоод доорх командуудыг оруулна. Квант 2-той Round Robin-оор A (5 тик), B (3), C (2) ээлжилж ажиллана. C нь t=5, B нь t=8, A нь t=9 тик дээр дуусна, өөрөөр хэлбэл дуусах хугацаа 6, 9, 10 болно. Дундаж хүлээлт 5.00 гарна. Дараа нь «Эхнээс нь» дарж бодлогыг FCFS болгоод ижил процессуудыг дахин оруулж хүлээлтийг харьцуулна (FCFS: 4.33). Командаар: `./kds examples/scheduler.txt`

```
OK sched policy rr q=2
OK proc new P1 A burst=5 (alive=1)
OK proc new P2 B burst=3 (alive=2)
OK proc new P3 C burst=2 (alive=3)
SCHED policy=rr(q=2) time=0 running=none ready=P1(A:5), P2(B:3), P3(C:2) done=0
TICK t=0 P1(A) remaining=4
TICK t=1 P1(A) remaining=3 preempted
TICK t=2 P2(B) remaining=2
TICK t=3 P2(B) remaining=1 preempted
TICK t=4 P3(C) remaining=1
TICK t=5 P3(C) remaining=0 finished
TICK t=6 P1(A) remaining=2
TICK t=7 P1(A) remaining=1 preempted
TICK t=8 P2(B) remaining=0 finished
TICK t=9 P1(A) remaining=0 finished
TICK t=10 idle
SCHED policy=rr(q=2) time=11 running=none ready=(empty) done=3
```

Жишээ 4 (хэш хүснэгт ба мод). «Тоглох» хуудсанд «hash» бүтцийг сонгоод «ab», «ba» зэрэг түлхүүр нэмэхэд тэд «bob»-той нэг bucket-д орж collision үүсэхийг харна. «tree» бүтцийг сонгоод эхлээд энгийн төрлөөр 1–7-г дараалуулан оруулна (өндөр 7). Дараа нь «Эхнээс нь» → төрөл «avl» болгож дахин 1–7-г оруулж харьцуулна (өндөр 3). Командаар: `./kds examples/search.txt`

```
OK hash put ab -> 7 (bucket=3, size=5, collision)
OK hash put ba -> 8 (bucket=3, size=6, collision)
HASH buckets=4 size=6 collisions=2: b0=[dave:11] b1=[carol:9] b2=[alice:1234] b3=
[bob:5678 -> ab:7 -> ba:8]
OK hash get ba = 8 (bucket=3, steps=3)
OK hash get zed = not found (bucket=3)
TREE size=5 height=5 inorder: 1, 2, 3, 4, 5
TREE(avl) size=7 height=3 inorder: 1, 2, 3, 4, 5, 6, 7
OK tree find 5 = yes
TREE(avl) size=6 height=3 inorder: 1, 2, 3, 5, 6, 7
```

1.8 Түгээмэл алдаа ба шийдэл

Мессеж	Шалтгаан	Шийдэл
`ERR list: day out of range for month`	Байхгүй огноо (31/4, 29/2/1900)	Зөв огноо оруулах
`ERR list: usage <op> <day> <month> <year>`	3 тоо биш (дутуу, үсэг, «2/8/1995» гэж налуу зураастай)	`list add_tail 2 8 1995`
`ERR queue: overflow (limit 8)`	8-аас олон push	pop хийх
`ERR queue: underflow (empty)`	Хоосон дээр pop	Эхлээд push
`ERR bitmap: index N out of range`	Индекс хэтэрсэн	0..n−1 дотор оруулах
`ERR tree: duplicate N`	Ижил утга	Өөр утга
`ERR tree: mode can only be changed …`	Мод хоосон биш	«Эхнээс нь» / бүх тоог хасах
`ERR hash: key and value must be …, key must be …`	12-оос урт эсвэл «-» гэх мэт тэмдэгттэй	Ж: alice, key_1
`ERR hash: full (limit 24)`	24 элемент	`hash del` эсвэл `hash clear`
`ERR proc: name must be …`	Нэр урт эсвэл тусгай тэмдэгттэй	Ж: A, job1
`ERR proc: too many processes`	8-аас олон амьд процесс	Зарим нь дуустал tick хийх
`ERR unknown structure 'x'`	Бүтцийн нэр буруу	list, queue, stack, bitmap, hash, tree, proc, sched
`insmod: Key was rejected by service`	Secure Boot, гарын үсэггүй модуль	MOK-оор гарын үсэг зурах (2.6)
`insmod: Invalid parameters`	`start` 1..1 000 000-аас гадуур	Ж: `start=27`
`make: *** /lib/modules/…/build: No such file`	Kernel header суугаагүй	`sudo apt install linux-headers-$(uname -r)`

2. Operational заавар

2.1 Repository clone хийх

```
git clone https://github.com/grl059/OS_BD1.git kernel-data-structures
cd kernel-data-structures
git checkout v1.0.0        # 9-р долоо хоногт нийтэлсэн хувилбар (GitHub Release)
```

2.2 OS, compiler/runtime, version

Хэрэгсэл	Туршсан хувилбар	Шаардлага
OS	Ubuntu 24.04 LTS	Linux / macOS / WSL (CLI); жинхэнэ Linux (module)
gcc	13.3	≥ 9 (ASan/UBSan дэмжлэгтэй); clang мөн болно
GNU make	4.3	—
Python	3.13	≥ 3.8 (fuzz, AVL тест)
Node.js	22	≥ 18 (веб апп, text==json тест, CI)
Linux kernel	6.11.0-17-generic	headers нь ажиллаж буй kernel-тэй ижил хувилбартай байх

2.3 Dependency суулгах

```
sudo apt update
sudo apt install build-essential python3 nodejs     # CLI ба тест
sudo apt install linux-headers-$(uname -r)          # зөвхөн kernel module-д
```

С цөм гадны сан (library) ашиглаагүй тул өөр dependency шаардлагагүй. Веб апп `web/package.json`-д заасан сангуудыг `npm install`-оор татна (интернэт шаардлагатай зөвхөн энэ алхамд).

2.4 Build ба Run команд

```
make                                  # ./kds (-Wall -Wextra -
fsanitize=address,undefined)
make SAN=0                            # sanitizer-гүй (ASan runtime ачаалагдахгүй
орчинд)
./kds examples/textbook.txt           # script
echo "queue push 5" | ./kds           # stdin-ээс
./kds --json examples/textbook.txt    # JSON горим (веб аппын хэрэглэдэг горим)
./kds                                 # интерактив, Ctrl+D-ээр гарна
make clean                            # build-ийн гаралтыг устгах

# Веб апп (эхлээд дээрх make-ээр ./kds-г build хийсэн байх ёстой):
cd web && npm install && npm run dev   # http://localhost:3000 дээр нээнэ
```

2.5 Үндсэн тестийн өгөгдөл

```
make test
#   PASS textbook / all_structures / errors / scheduler / scheduler_full / search
#   PASS stress (5000 x 8 commands, ERR 0, no sanitizer report)
make check                              # make test + доорх гурав
python3 tests/json_check.py             # текст горим == --json горим, 6 скрипт: PASS x6
python3 tests/fuzz_json.py 8 5000       # 8 seed x 5000 санамсаргүй команд (тэгш seed-д
AVL): PASS x8
python3 tests/avl_check.py              # AVL: inorder ба өндөр <= 1.44*log2(n+2), 30
гүйлт: PASS
```

Хүлээгдэх гаралт `tests/expected_*.out`-д байна. Шинэ тест нэмэхдээ `examples/нэр.txt` үүсгээд `./kds examples/нэр.txt > tests/expected_нэр.out` гэж хадгалж, гаралтыг гараар шалгаад `tests/run_tests.sh`-ийн жагсаалтад нэрийг нэмнэ.

2.6 Нэмэлт boundary/error test

`examples/errors.txt`: буруу огноо (31/4, 29/2/1900), сар 13, жил 1800, үсэг бүхий аргумент, хоосон бүтцээс pop, bitmap size 0 ба 129, индекс −1 ба 99, давхардсан tree утга, байхгүй утга устгах. `examples/scheduler.txt`-ийн төгсгөл: урт нэр, ажил 0 ба 21, квант 0 ба 11, мэдэгдэхгүй бодлого. `examples/scheduler_full.txt`: 9 дэх процесс татгалзагдана. `examples/search.txt`: 13 тэмдэгттэй түлхүүр, «-» тэмдэгттэй түлхүүр, хоосон биш модны төрлийг солих. Эдгээрийн мөр бүр ойлгомжтой `ERR` мессеж өгч, програм үргэлжлэн ажиллах ёстой.

2.7 Kernel module

```
sh src/kernel/run_module_test.sh
#   make -> docs/kernel_build_log.txt
#   PASS insmod  (start=25, 24 гишүүн)
#   PASS insmod start=15 (18 гишүүн)
#   PASS insmod start=0 (fail: Invalid parameters)
#   -> docs/kernel_dmesg.txt

# Гараар:
cd src/kernel && make
sudo insmod kds_module.ko start=15 && sudo dmesg | grep kds:
sudo rmmod kds_module && sudo dmesg | grep kds: | tail -30
```

Хүлээгдэх dmesg (`start=15`): «loading module (start=15)», 4 «color red=… blue=… green=…» мөр, 18 «collatz» мөр (15 46 23 70 35 106 53 160 80 40 20 10 5 16 8 4 2 1), «collatz start=15 terms=18», 5 «birthday d/m/y», «fifo out» 10, 20, 30, «bitmap weight=2, bit15=1». rmmod үед «removing color/collatz/birthday» мөрүүд болон «module unloaded» гарна. `start=0` үед «start must be 1..1000000 (got 0)» гарч, insmod амжилтгүй болно.

Secure Boot идэвхтэй үед (`insmod: Key was rejected by service`) дараах тохиргоог нэг удаа хийнэ:

```
mkdir -p ~/mok && cd ~/mok
openssl req -new -x509 -newkey rsa:2048 -keyout MOK.priv -outform DER -out MOK.der
\
        -nodes -days 3650 -subj "/CN=KDS module signing/"
sudo mokutil --import MOK.der       # reboot хийж, MOK дэлгэц дээр Enroll MOK
# build бүрийн дараа:
sudo /usr/src/linux-headers-$(uname -r)/scripts/sign-file sha256 ~/mok/MOK.priv
~/mok/MOK.der kds_module.ko
```

`MOK.priv` түлхүүрийг repository-д хэзээ ч commit хийхгүй (`.gitignore`-д хассан).

2.8 Хүлээгдсэн үр дүн

Команд	Хүлээгдэх
`make`	Анхааруулгагүй compile хийгдэж `./kds` үүснэ
`make test`	7 PASS (6 скрипт + stress), exit 0
`make check`	Нийт 53 PASS (7 + 6 text==json + 8 fuzz + 32 AVL), FAIL 0
Веб апп (npm run dev)	Browser-ийн console-д алдаа гарахгүй; «Тоглох» хуудасны гаралт CLI-тай ижил (хоёулаа нэг C программ ашигладаг)
`run_module_test.sh`	3 PASS, build log ба dmesg файл үүснэ

2.9 Known issues

- Kernel module WSL, Docker-т ачаалагдахгүй (kernel header ба insmod эрх байхгүй). Linux VM эсвэл үндсэн Linux ашиглана.
- Өөр kernel хувилбар дээр API өөрчлөгдсөн бол module-ийг дахин build хийнэ.
- CLI-д нэг мөр 1023 тэмдэгтээс урт бол «ERR line too long» гарна.
- Веб апп серверийн хажуу (`web/src/app/api/run/route.ts`) C цөмийг дэвсгэр дээр дуудаж ажиллуулдаг тул веб аппыг нээхийн өмнө `make`-аар `./kds` (эсвэл Windows дээр `kds.exe`) build хийсэн байх ёстой; эс бөгөөс «C core not built» алдаа гарна.
- Hash bucket-ийн тоо тогтмол; queue, stack, процесс 8, hash 24 элементээр хязгаарлагдсан (дүрслэлд багтаах зорилгоор).
- **macOS 26 + Apple clang:** ASan runtime нь програм эхлэх үед төгсгөлгүй хүлээлт үүсгэж болно (гаралтгүй, 100% CPU). Иймд үед `make SAN=0` гэж sanitizer-гүй build хийнэ; тест болон веб апп хэвийн ажиллана, зөвхөн ASan/UBSan шалгалт хийгдэхгүй.